DATA ANALYTICS

Google Colab (Data Wrangling/Preprocessing/EDA)

GETTING STARTED

Introduction to Google Colab

Your free cloud-based Python coding environment

Google Colab (short for Colaboratory) is a free tool from Google that lets you write and run Python code in your browser no installation needed. It's the same environment professional data scientists use, and everything we do in this course will run here.

Step 1: Open Google Colab

Open your browser and go to:

```
colab.research.google.com
```

Sign in with your Google account. You'll see the Colab welcome screen.

> 💡 **Tip:** You can also access Colab directly from Google Drive. Just go to drive.google.com, click + New -> More -> Google Colaboratory.

Step 2: Create a New Notebook

A notebook is your coding workspace. Think of it like a Word document, but with live code inside it.

- Click File in the top menu bar
- Select New notebook
- A new tab opens this is your blank notebook!

You'll see a blank cell at the top. This is a code cell where you type your Python code.

Step 3: Understand the Interface

Part of the Screen	What it does
▶️ `Play button`	Runs the code in a cell. You can also press Shift + Enter on your keyboard.
`+ Code button`	Adds a new code cell below. Use this to separate your code into logical chunks.
`+ Text button`	Adds a text cell for notes and explanations (no code runs here).
`File menu`	Save your notebook, download it, or share it with your teacher.
`Runtime menu`	If your code gets stuck, click Runtime -> Restart session to start fresh.

Step 4: Run Your First Cell

Let's make sure everything works. Click on the empty code cell and type this:

```
print("Hello, Data Science!")
```

Press the ▶️ Play button (or Shift + Enter). You should see the output appear just below the cell:

```
Hello, Data Science!
```

🎉 **It works!** If you see that output, your notebook is running correctly. Every time you change code, just press ▶️ again to re-run it.

Step 5: Import Pandas

Pandas is the Python library we use to work with data. Colab has it pre-installed we just need to import it at the top of our notebook. Create a new code cell and type:

```
import pandas as pd
print('Pandas is ready!')
```

Run the cell. You only need to import Pandas once per session, at the very top of your notebook.

Step 6: Upload the Video Game Sales CSV File

Now for the important part loading our dataset. We need to upload the CSV file from your computer into Colab.

Method A: Upload via the Files Panel (Recommended for Beginners)

- Click the folder icon 📁 on the left side panel to open the Files browser
- Click the Upload icon (a page with an upward arrow) at the top of the Files panel
- Find and select your vgsales.csv file on your computer
- Wait for the upload to complete you'll see the file appear in the panel

Once uploaded, load it into your notebook with this code:

```
df = pd.read_csv('vgsales.csv')
print('Dataset loaded!')
print(df.shape)    # Shows (rows, columns)
```

Method B: Upload Using Code (Faster for Large Files)

Alternatively, you can use a built-in Colab widget to upload files directly from a code cell:

```
from google.colab import files

uploaded = files.upload()    # A file picker will appear
```

After running this cell, a Choose Files button will appear. Click it, select your CSV, and wait for the upload. Then run:

```
df = pd.read_csv('vgsales.csv')
df.head()    # Preview the first 5 rows
```

⚠️ **Important:** Every time you close Colab and reopen it, uploaded files are deleted. You'll need to re-upload your CSV at the start of each session. To avoid this, consider saving the CSV to your Google Drive (see Step 7 below).

Step 7 (Optional): Load from Google Drive

If you're tired of re-uploading the file every session, you can save it to your Google Drive once and load it from there:

```
from google.colab import drive
drive.mount('/content/drive')   # Follow the authorization steps

# Then load your file using the full path:
df = pd.read_csv('/content/drive/MyDrive/vgsales.csv')
```

Colab will ask you to authorize access to your Google Drive. Click the link, sign in, and paste the code it gives you. After that, your Drive files are always available!

Step 8: Verify Your Data Loaded Correctly

Run these two cells to confirm everything looks right before starting any analysis:

```
# Check the shape: (rows, columns)
print(df.shape)

# Preview the first 5 rows
df.head()
```

You should see something like (16598, 11) meaning 16,598 games and 11 columns. If you see an error, check that the filename matches exactly (including uppercase/lowercase letters).

🔍 **Common Error:** FileNotFoundError: Make sure the CSV filename in your code exactly matches the uploaded filename. It's case-sensitive! 'vgsales.csv' is not the same as 'VGSales.csv'.

PART 1 • WEEK 2

Data Wrangling & Preprocessing

Focus: "Garbage In, Garbage Out" – Cleaning Data

Real-world data is messy. If we run our analysis on messy data, our insights will be completely wrong. Last week, we talked about how to spot these errors on paper. Today, we are going to use Python's Pandas library to automate the cleanup of our Video Game Sales dataset.

1. Handling Missing Values

Imputation vs. Deletion

Look at the dataset. Super Mario Bros. has missing critic scores because Metacritic didn't exist in 1985. In Pandas, missing values show up as NaN (Not a Number).

We cannot do math on a missing value, so we have two main strategies:

Strategy	What it does	Python Code
Deletion	Throws away entire rows or columns with missing data.	`df.dropna(subset=['Critic_Score'])`
Imputation	Fills in blanks with a calculated guess (mean or median).	`df['Critic_Score'].fillna(df['Critic_Score'].mean())`

💡 **When to use which?** Use Deletion when you have plenty of data and missing bits aren't crucial. Use Imputation when you have a small dataset and can't afford to lose rows.

2. Data Formatting, Normalization & Type Conversion

Sometimes numbers are disguised as text, or scales are mismatched. Here's how to fix both:

Type Conversion

If a user score is entered as "tbd" (To Be Determined), Python treats that entire column as text. We have to force it back into numbers using:

```
df['User_Score'] = pd.to_numeric(df['User_Score'], errors='coerce')
```

> 📝 **Note:** The errors='coerce' argument turns any non-numeric values into NaN automatically very useful!

Normalization

Our Critic_Score is out of 100, but our User_Score is out of 10. To compare them fairly, we need to normalize them to the same scale:

```
df['User_Score_Normalized'] = df['User_Score'] * 10
```

3. Filtering, Sorting & Grouping

No more shuffling paper! Here is how we ask our dataset specific questions using Pandas:

Filtering Show me only Nintendo games:

```
nintendo_games = df[df['Publisher'] == 'Nintendo']
```

Sorting What is the highest-selling game globally?

```
top_games = df.sort_values(by='Global_Sales', ascending=False)
```

Grouping What are the total global sales per Genre?

```
genre_sales = df.groupby('Genre')['Global_Sales'].sum()
```

PART 2 • WEEK 3

Exploratory Data Analysis (EDA)

Focus: Statistical Analysis & Pattern Recognition

Now that our data is clean, we can explore it to find hidden patterns. EDA is the detective work of Data Science we're looking for clues in the numbers.

🔍 **Key Idea:** EDA is about asking questions and letting the data answer them. We use statistics and visualizations to uncover patterns we didn't know existed.

1. Descriptive Statistics

Descriptive statistics summarize the main features of a dataset. Instead of calculating these by hand, Pandas does it in one line:

```
df.describe()
```

Behind the scenes, Python is calculating all of the following for you:

- **Mean:** Mean

The mathematical average of all values in a column.

- **Median:** Median

The exact middle number when all values are sorted. This is often better than the mean when you have extreme outliers like Wii Sports selling 82 million copies!

- **Mode:** Mode

The most frequently occurring value (e.g., the most common Genre in the dataset).

- **Variance:** Variance

How much the data points spread out from the mean. Calculated as:

$$s^2 = \Sigma(x_i - \bar{x})^2 / (n - 1)$$

- **Standard Deviation (SD):** Standard Deviation (SD)

The square root of the variance. It tells us, on average, how far each data point is from the mean. Calculated as:

$$s = \sqrt{[\Sigma(x_i - \bar{x})^2 / (n - 1)]}$$

2. Univariate vs. Multivariate Analysis

When running EDA, we have to ask: how many variables are we looking at at once?

Univariate Analysis (One Variable)

We look at a single column to understand its distribution. Example question:

> ? **Example:** What is the distribution of Critic Scores across all games?

We use tools like histograms or box plots for this type of analysis.

Multivariate Analysis (Two or More Variables)

We look at how variables interact with each other. Example question:

> ? **Example:** Do higher Critic Scores lead to higher Global Sales?

We use tools like scatter plots to visualize these relationships.

3. Correlation vs. Causation

When we plot Critic_Score against Global_Sales on a scatter plot, we might see a positive trend as scores go up, sales tend to go up. This is called a Positive Correlation.

> ⚠ **Golden Rule of Data Science:** Correlation does NOT imply Causation. Just because two things move together doesn't mean one causes the other!

A high Metacritic score doesn't magically force people to buy a game. The reality is a hidden third variable:

Game Quality -> earns good reviews (high Critic Score)

Game Quality -> generates good word-of-mouth -> high sales

Both the high score AND the high sales are effects of underlying quality. This is one of the most important concepts in all of Data Science, always ask "What else might explain this?"

Lecture & Lab Activity 4

LEC_ACT – 4 >> Google Colab Basics

Open Google Colab, upload the Video Game Sales CSV, and run each code block in order: from importing Pandas all the way through the grouping and EDA sections. Make sure every cell runs without errors before moving on to the next one.

Then, answer the following questions on a sheet of paper:

1. When you ran df.shape, how many rows and columns did your dataset have? What does each number represent?
2. After filtering for GB and NES games, how many rows are left in your retro_data DataFrame? Why did the number shrink?
3. You filled missing Critic_Score values with 0. Do you think that was a fair choice? What else could you have used instead?
4. Look at the Mean vs. Median for Global_Sales. Which one is higher, and what does that tell you about the dataset?
5. After grouping by Genre, which genre had the highest average global sales? Does that result surprise you — why or why not?
6. Think about the Critic Score vs. Global Sales relationship. Can you think of a real game that had a high critic score but low sales, or vice versa? What might explain that?

Upload the following files:

1. Ipynb file
2. PDF containing screenshots of your google colab outputs
3. PDF containing picture of your answers

LAB_ACT – 5 >> Google Colab Demonstration

Record a short video of yourself going through your completed Colab notebook.

For each code block, you need to do three things: show the code you wrote, run it live on screen, and explain out loud what the code does and what the output means. You must show your camera clearly with a clear voice beside the colab screen (webcam should not be small).

You will also explain here your answers for the 6 questions as you demonstrate your code.

Quick Reference Cheat Sheet

Week 2 Key Pandas Commands

```
df.dropna(subset=['column'])          # Delete rows with missing values
df['col'].fillna(df['col'].mean())    # Impute with mean
pd.to_numeric(df['col'], errors='coerce')  # Type conversion
df[df['col'] == 'value']              # Filter rows
df.sort_values(by='col', ascending=False)  # Sort
df.groupby('col')['other'].sum()      # Group & aggregate
```

Week 3 Key Statistical Concepts

Term	Definition	Python Code
Mean	Mathematical average	`df['col'].mean()`
Median	Middle value when sorted	`df['col'].median()`
Mode	Most frequent value	`df['col'].mode()`
Variance	Spread from the mean (squared)	`df['col'].var()`
Std. Deviation	Average distance from mean	`df['col'].std()`
df.describe()	All stats in one shot!	`df.describe()`